

Java

a

API: Interfaz de Programación de Aplicaciones (Application Programming Interface) da los medios para desarrollar aplicaciones Java, proporciona un conjunto de clases e interfaces predefinidas, que se copian durante la instalación de Java, para realizar aplicaciones, y utilizarlas cuando sea necesario.

Organizada en paquetes que contienen un conjunto de clases relacionadas semánticamente. Se importa a la clase para poder utilizar los recursos.

```
import nombre.paquete.*;
import nombre.paquete.NombreClase; //solo importa la clase nombrada.
```

Array: Estructuras de datos fija que almacenan varios elementos del mismo tipo en posiciones consecutivas de memoria. Cada elemento puede ser de tipo simple como caracteres, entero, real, o de tipo compuesto, estructurado como son otros arrays u objetos.

Tamaño fijo (se define al crearlos y no se puede cambiar).

El índice inicial de los arrays en Java es 0. Se accede con índices (array[0], array[1], etc.).

```
int[] numeros = {10, 20, 30};
System.out.println(numeros[1]); // Imprime 20
```

Matriz: Estructura bidimensional, array de arrays.

Uso de [ren][col]:

ren = fila (row)

col = columna (column)

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(matriz[1][2]); // 6 (fila 1, columna 2)
```

Declaración

```
<tipo dato> [ ] <nombre_array>;
<tipo dato> <nombre_array> [ ];
<tipo dato> [ ] <nombre_array> [ ];
```

Ej.

```
int [ ] arrayEnteros; //Declaración array de enteros
string myStringArray [ ]; //Declaración array de strings
char [ ] myCharArray [ ]; //Declaración array de chars
```

Inicializar

Por tamaño:

```
<tipo dato> [ ] <nombre_array> = new <tipo dato>[ tamaño];
```

Java

```
Ej. int[ ] arrayEnteros = new int[ 5 ];
```

Con valores:

```
<tipo dato> [ ] <nombre_array> = { val1, val2, ... };
```

```
Ej. String[ ] arrayNombres = { "Juan", "María", "Susana" };
```

Recuperar o asignar un valor a una posición uso del índice.

Recuperar:

```
<tipo dato> <nombre_variable> = <nombre_array> [ índice ];
```

```
Ej. int numero = arrayEnteros [ 2 ];
```

b

Bloques: Conjunto de sentencias, que están delimitados por llaves ({}).

c

Casting: (función Casting): transforma un **dato primitivo** de un tipo a otro en función del tamaño que ocupa. La conversión puede ser de dos formas:

Casting implícito: cuando el **contenedor es más grande** que el valor a almacenar.

bite-short-char-int-long-float-double

Casting explícito: cuando el **contenedor es más pequeño** que el valor a almacenar. En este caso, puede que haya pérdida de datos.

double-float-long-int-char-short-byte

Clase: Todo programa Java se llama clase. Plantilla (blueprint) que define la forma que tendrán los objetos creados en la misma. Es decir el tipo de características (atributos) y los comportamientos (métodos) comunes.

Atributos: Almacenan **características** de un objeto (lo que se sabe de un objeto; ej. color, longitud). El nombre por convención se escribe empezando por **minúscula**, y delante de cada atributo es necesario indicar el **tipo de dato** seguido del **nombre del atributo**.

Datos

Primitivo: int (números enteros), float (decimales de hasta 7 dígitos), double (decimales más precisos de hasta 16 dígitos).

Complejo: String (texto o cadenas de texto), Date(fecha). También podemos crear nuestros propios tipos complejos.

Métodos: Representan el **comportamiento** de un objeto (lo que hace un objeto; ej. acelerar, frenar).

Java

Declaración

Mediante la palabra reservada **class** (puede ir precedida de un modificador de acceso) seguida de su **nombre**.

Modificadores

public: atributo público, se puede acceder a la clase desde **cualquier lugar del programa**.

protected: son accesibles directamente sólo desde la propia clase, de las subclases de esta (herencia) y las clases que se encuentran dentro del mismo paquete es decir se puede acceder a la clase desde el **paquete o subclase** de esta.

private: atributo privado no se podrá acceder directamente a él desde otra clase distinta. Por tanto, será necesario recurrir a algún método que nos proporcione o nos modifique su valor. sólo se puede acceder a la clase desde esta **misma**.

```
[modificadores] class NombreClase{
```

Implementación

Incluye los atributos y métodos de la clase, después de su declaración.

```
public class Coche{  
  
    //declaración de atributos  
    private String color;  
    private double longitud;  
  
    //declaración de métodos  
    public void acelerar(){};  
    public void frenar(){};  
}
```

Atributos

Comentario:

Comentarios de una sola línea

```
// Esto es un comentario
```

Comentarios multilínea

```
/* Esto es un comentario  
de una o más líneas */
```

Comentarios de documentación para la herramienta Javadoc

```
/** Esto es un comentario para javadoc */
```

Clase Wrapper: envoltorio, permite usar tipos primitivos (int, char, boolean, etc.) como objetos.

int → **Integer**

double → **Double**

char → **Character**

Java

boolean → Boolean

Permiten usar métodos útiles y almacenar valores primitivos en colecciones que sólo aceptan objetos (como `ArrayList<Integer>`).

```
int num = 5;
Integer objNum = Integer.valueOf(num); // Boxing (convertir primitivo a objeto)
int num2 = objNum.intValue();          // Unboxing (objeto a primitivo)
```

Constante: Objeto que no experimenta cambios durante todo el desarrollo del algoritmo.

Declaración

Definir a nivel de clase (global)

Hacer uso de 2 palabras reservadas: **static** y **final**

Nombres en mayúsculas.

```
static final <tipo_dato> <NOMBRE CONSTANTE> = valor;
```

Constructor: Método especial que inicializa los atributos del objeto. Establece el estado inicial del objeto.

- Debe tener el mismo nombre de la clase.
- No tiene tipo de retorno (ni `void`).
- Debe declararse como `public`
- Son llamados automáticamente cuando se crea un objeto.

En todas las clases Java, siempre hay un constructor. (Si no lo define el usuario, Java proporciona automáticamente el constructor por defecto o implícito: constructor vacío).

Una vez definido un constructor, el constructor por defecto ya no se usa.

Inicializan atributos de la clase cada vez que se crea un objeto con valores coherentes.

Class

```
public class Persona {
    //atributos que va a inicializar
    String nombre;
    int edad;

    // Constructor
    public Persona(String nombre, int edad){
        this.nombre = nombre;
        this.edad = edad;
    }
    public void mostrarDatos (){
        System.out.println("El nombre es: "+ nombre);
        System.out.println("La edad es: "+ edad);
    }
}
```

Java

Main

```
public class Main {  
    public static void main (String [] args){  
        Persona p1 = new Persona("Ana", 21);  
        p1.mostrarDatos();  
    }  
}
```

Explicación de `Persona p1 = new Persona("Ana", 21);`

`Persona` → tipo de dato (en este caso una clase).

`p1` → nombre de la variable para guardar el objeto.

`new` → crea un objeto nuevo en memoria.

`new Persona("Ana", 21)` → es la creación (instanciación) de un objeto de tipo `Persona` usando el constructor definido.

Es decir creación de un objeto de la clase `Persona` con nombre `Ana` y edad `21`, guardarlo en la variable `p1`

*Java convierte automáticamente un `int` a `String` cuando está junto a un `+` con texto.

"Edad: " + 21 // se convierte en "Edad: 21"

int (número entero), se escriben sin comillas. ("21", Java lo entendería como un texto)

this: hace referencia a las propiedades (atributos) del objeto, también se usa para acceder a otros métodos o constructores del objeto actual:

`this.atributo`: accede a un atributo de la misma clase.

`this.metodo()`: accede a un método de la misma clase.

`this()`: llamamos a otro constructor de la misma clase.

Sólo es necesario en el momento en que asignamos a un atributo un parámetro en el que coincida con el nombre, así el compilador puede distinguir si hace referencia al atributo o al parámetro, si el atributo y el parámetro tienen distinto identificador no haría falta.

Super: acceder a atributos, métodos y constructores de la clase padre desde la clase hija.

- Vacío/default

No tiene parámetros de entrada ni instrucciones a ejecutar.

Sirve para crear un objeto "vacío", con valores iniciales por defecto (`null` para objetos, `0` para números, `false` para booleanos).

Instanciar primero y llenar después.

En java

```
public Persona () {  
}
```

En main

```
Persona p1 = new Persona();  
System.out.println(p1.nombre); // Sin nombre
```

Java

```
System.out.println(p1.edad);    // 0

// posteriormente modificarlo
p1.nombre = "Ana";
p1.edad = 21;
```

- De copia

Inicializa (duplica) un objeto nuevo copiando los datos de otro. Evita que ambos usen el mismo espacio en memoria (evita aliasing).

Crear un objeto nuevo a partir de un segundo objeto ya existente de la misma clase (único parámetro de entrada).

Copia los atributos del objeto original. Asignar a cada uno de los atributos el valor de los atributos del objeto a copiar.

En main

```
Persona p1 = new Persona("Ana", 21);
Persona p2 = new Persona(p1); // usa el constructor de copia

System.out.println(p2.nombre); // Ana
System.out.println(p2.edad);   // 21
```

Sobrecarga de constructores

Permite crear objetos con diferentes niveles de detalle.

Flexibiliza la inicialización.

Desde main con **new** crea un objeto y busca un constructor (en la clase definida -otro archivo) que coincida con el número y tipo de parámetros.

Si lo encuentra, lo usa.

Si no lo encuentra, da error.

Si hay varios parecidos, elige el más compatible con los tipos de datos que tiene.

Collection: superinterfaz para todos los tipos de colecciones (List, Set, Queue...).

Arrays es fijo entonces usar List, ArrayList, Set - HashSet, Map-HashMap

Define operaciones comunes (añadir, eliminar, verificar si contiene un elemento).

```
Collection<Integer> coleccion = new ArrayList<>();
coleccion.add(5);
coleccion.add(10);
```

* **Integer:** clase envoltorio (wrapper) que permite usarse en colecciones (List<Integer>). Usar métodos como Integer.parseInt().

Java

Module: java.base **Package:** java.util

Interface **list <E>**: **java.util** define una lista ordenada (índice) con elementos duplicados.

***E** = **Element**. Parámetro de tipo genérico, se reemplaza con el **tipo de dato** se quiera guardar en la lista.

Mantiene orden de inserción.

Acceso: por índice (get, indexOf).

```
List<String> lista = new ArrayList<>();  
lista.add("A");  
lista.add("B");  
System.out.println(lista.get(0)); // A
```

Array list: clase que implementa `List` usando un array dinámico.

Mantiene orden (índice)

Elementos agregados al final

Permite elementos duplicados

```
List<Integer> lista = new ArrayList<>();  
lista.add(100);
```

InterfaceSet<E>: Colección sin elementos duplicados.

HashSet: Implementación de set para guardar elementos sin orden garantizado.

No mantiene orden (índice)

No permite elementos duplicados (no lo agrega, reemplaza)

Utiliza hashCode: código único public int

Acceso: Iterator / for-each

```
Set<String> set = new HashSet<>();  
set.add("A");  
set.add("A"); // No se agrega duplicado
```

Interface Map <K,V>: interfaz que almacena datos en pares clave-valor, donde cada clave (es única y se asocia a un valor (puede repetirse).

***K** = Key (clave), **V** = Value (valor).

HashMap: implementación de `Map` basada en una tabla hash. No garantiza orden de los elementos, pero permite búsqueda, inserción y eliminación muy rápidas.

Pares de llave/valor

No permite llaves duplicadas (valores si)

Una llave solo tiene un valor

Acceso: get(clave)

```
Map<String, Integer> edades = new HashMap<>();  
edades.put("Ana", 25);  
edades.put("Luis", 30);  
System.out.println(edades.get("Ana")); // 25
```

Métodos útiles para list, set y map

Método	Descripción	Tipo de retorno	Aplica en
<code>.size()</code>	Devuelve el número de elementos en una colección o lista.	int	List, Set, Map (Map cuenta pares)
<code>.isEmpty()</code>	Devuelve true si la colección no contiene elementos.	boolean	List, Set, Map
<code>.get(int index)</code>	Devuelve el elemento en la posición indicada por el índice	Tipo de la lista	List
<code>.indexOf(obj)</code>	Devuelve el índice de la primera ocurrencia del elemento o -1 si no existe.	int	List
<code>.lastIndexOf(obj)</code>	Devuelve el índice de la última ocurrencia del elemento o -1 si no existe.	int	List
<code>.contains(obj)</code>	Devuelve true si la colección contiene el elemento indicado.	boolean	List, Set, Map (containsKey / containsValue)
<code>.add(obj)</code>	Agrega un elemento a la colección (o clave-valor en Map usando .put).	boolean/void	List, Set
<code>.remove(int text)</code>	Elimina un elemento.	Tipo de dato	List, Set, Map
<code>.remove(obj)</code>	Elimina un elemento (o clave en Map).	boolean/void	List, Set, Map
<code>.clear()</code>	Elimina todos los elementos de la colección.	void	List, Set, Map
<code>.put(key, value)</code>	Inserta o reemplaza un par clave-valor en un Map.	V (valor previo)	Map
<code>.get(key)</code>	Devuelve el valor asociado a la clave.	Valor del mapa	Map
<code>.values()</code>	Devuelve una colección con todos los valores del mapa.	Collection<V>	Map
<code>[ren][col]</code>	acceso por fila y columna	Tipo de dato	Matriz

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/module-summary.html>

d

Datos:

Primitivos o simples: entidades elementales como números, caracteres o Verdadero/Falso.

- **Int:** números de tipo entero, como años o días de la semana.

```
int nAño = 2005;
```

- **Float:** números de tipo decimal de hasta 7 dígitos; como altura o peso.

```
float nAltura = 74.54;
```

- **Double:** números de tipo decimal muy precisos, hasta 16 dígitos; como área o longitud.

```
double nLongitud = 2500000.5497;
```

- **Char:** variables de tipo letras y números, como la letra "A".

```
char unCaracter = 'A'; //Siempre comillas simples
```


Java

	Numeros Enteros	Números reales con punto decimal	Valor único
8b	byte		
16b	short		char ''
32b	Int	float F	
64b	long L	double	

• **Boolean:** datos de tipo lógico que sólo pueden ser verdadero o falso; por ejemplo, si es par o impar, positivo o negativo...

```
boolean par = true;
boolean positivo = false;
```

Referencia o complejos: pueden estar formados por varias variables, otros datos complejos y/o funciones. Son objetos, y necesitan ser creados por el programador.

El más utilizado es el tipo de dato String, permite manejar textos y cadenas de texto.

Ej. concatenar 2 tipos de datos "Hola" y "Fernando". El operador "+" permite concatenar cadenas y obtener el resultado esperado "Hola Fernando":

```
String str1= "Hola";
String str2= "Ana";
String str3= str1+" "+str2; //str3 sería "Hola Ana"
```

e

Eclipse



Debug y Breakpoints

Colocar/Quitar breakpoint

- Doble click en el margen izquierdo de la línea (gutter) o
- Menú: Run > Toggle Breakpoint
- Atajo (Mac): ⌘⇧B (o simplemente clic en el margen)

Controles durante el debug (vista Debug)

- **Resume** (seguir): F8
- **Step Over** (siguiente línea sin entrar a métodos si es un método de una librería, Eclipse manda al código de la API (el .class).): F6
- **Step Into** (entrar a método): F5
- **Step Return** (salir del método actual): F7
- **Terminate** (detener): botón rojo cuadrado
- Cambiar perspectiva con los recuadros de la derecha superior

Vistas útiles durante debug

- Variables: ver valores actuales
- Expressions: agrega expresiones/variables para "watch"
- Breakpoints: lista y activación selectiva de breakpoints

Java

- Display: evalúa snippets rápidos (selecciona código → Display → Ctrl+U en Windows; en Mac usa el menú)

Breakpoints condicionales

- Clic derecho sobre el breakpoint → Breakpoint Properties...
- Marca **Conditional** y escribir la condición (ej. `i == 5`) útil para loops.



Formateo y limpieza de código

Formatear archivo (auto-indent, llaves, espacios, etc.)

- Menú: Source > Format
- Atajo (Mac): ⌘⇧F
- Pro tip: Ajusta el formato en Preferences > Java > Code Style > Formatter.

Organizar imports (añade/quita y ordena)

- Menú: Source > Organize Imports
- Atajo (Mac): ⌘⇧O

Quick Fix (correcciones rápidas)

- Coloca el cursor en el error → Menú: Edit > Quick Fix
- Atajo (Mac): ⌘1
- Ejemplos: crear método/constructor faltante, importar clase, cambiar tipo, etc.

Clean Project

- Menú: Project > Clean... (reconstruye desde cero; útil tras mover/renombrar cosas)

Build Automatically

- Menú: Project > Build Automatically (déjalo activado para detectar errores al vuelo)



"Close Unrelated Projects" (acelera y reduce ruido)

- Menú: Project > Close Unrelated Projects
- O cierra manualmente con clic derecho sobre proyectos que no uses → Close Project.



"Heredar" al crear una clase (el "Browse" del asistente)

Crear una nueva clase que **extiende** a otra:

1. File > New > Class (o Package Explorer → clic derecho → New > Class)
2. **Name:** escribe el nombre de tu clase (p. ej. Empleado)
3. **Superclass:** por defecto es `java.lang.Object`.
 - Haz clic en **Browse...**
 - Selecciona la clase padre (p. ej. Persona)
 - Eclipse rellenará `extends Persona`
4. Opcional: marca
 - **public static void main(String[] args)** si quieres método main
 - **Constructors from superclass** para que genere constructores que llamen a `super(...)`

Java

- **Inherited abstract methods** si heredaste de una clase abstracta (autogenera los @Override obligatorios)



Navegación shortcuts

- **Ir a la declaración:** F3 o ⌘+clic sobre el símbolo
- **Abrir tipo/clase por nombre:** ⌘⇧T
- **Abrir recurso por nombre (cualquier archivo):** ⌘⇧R
- **Renombrar seguro (refactor):** Alt+⌘+R (cambia nombre en todas las referencias)
- **Mover/extraer métodos/campos (refactor):** menú Refactor sobre la selección
- **Problems view:** Window > Show View > Problems (doble clic salta al error)

Notas rápidas para herencia y super en Eclipse

- Tras crear Empleado que **extiende** de Persona, si falta constructor en Empleado y el de Persona no es vacío, Eclipse te marca error → usa **Quick Fix** (⌘1) → Add constructor y genera la llamada a super(...).
- Si sobrescribes métodos: escribe el nombre y parámetros, o usa **Quick Fix** sobre la clase → Add unimplemented methods.
- Para insertar anotaciones @Override automáticamente, activa:
Preferences > Java > Code Style > Clean Up → configura y luego Source > Clean Up.

Mini checklist para “agilizar errores de sintaxis”

- Cierra proyectos que no usas: Project > Close Unrelated Projects
- Activa Build Automatically
- Corre Project > Clean... si algo se corrompe
- Usa Source > Organize Imports (⌘⇧O) y Source > Format (⌘⇧F)
- Corre Quick Fix (⌘1) sobre cada marcador rojo/amarillo
- Revisa Problems view y filtra por tu proyecto

Encapsulamiento: ocultar el estado interno de un objeto y exponer solo lo necesario mediante una interfaz pública (métodos).

Objetivo: proteger datos, evitar estados inválidos, permitir cambios internos sin romper a quien usa tu clase.

- Campos privados (`private`)
- Acceso controlado vía getters/setters
- Validaciones dentro de los setters (o en el constructor)
- Inmutabilidad cuando conviene (sin setters)
- Copias defensivas para tipos mutables (listas, fechas)

Java

Modifier	Class	Package	Subclass	World
Public	Green	Green	Green	Green
Protected	Green	Green	Green	Red
No modifier default	Green	Green	Red	Red
Private	Green	Red	Red	Red

*"No modifier" o default access modifier (*package-private*) uno de los 4 niveles de acceso. Si se declara una clase, atributo o método **sin** poner **public**, **private** o **protected**, Java lo interpreta con **default access**, es decir accesible solo dentro del mismo paquete. No accesible desde fuera del paquete, ni siquiera por herencia. Útil para "agrupar" clases/métodos de uso **solo internamente** en módulo propio, sin exponerlos como API pública.

Errores

1. **De sintaxis** → errores al escribir código incorrecto. Se detectan antes de compilar.
2. **De ejecución (runtime/exception)** → ocurren al correr el programa. Se manejan con **try-catch**:

```
try {  
    int num = Integer.parseInt("abc");  
} catch (NumberFormatException e){  
    System.out.println("Error: formato inválido");  
}
```

3. **De lógica** → el programa corre, pero da resultados incorrectos. Uso de debug sirve para encontrar errores de lógica o verificar si un método hace lo esperado.

Debug: proceso de ejecutar el programa paso a paso para analizar qué ocurre en memoria.

Breakpoint: una marca en una línea de código donde el programa se detiene para inspeccionar variables y flujo.

printStackTrace: Método de Throwable que imprime en **stderr** la ruta de llamadas (stack) hasta el punto donde ocurrió la excepción: clase, método, línea.

Usarlo en desarrollo para diagnosticar, no como mensaje final al usuario.

`e.getMessage()` → solo el **mensaje** ("/ by zero").

`e.printStackTrace()` → mensaje + **pila completa**.

```
public static void demo() {  
    try {  
        Calculable malo = new Rectangulo("X", 10, -5); // supón que validas altura > 0  
        System.out.println(malo.calcularArea());  
    } catch (IllegalArgumentException e) {
```

Java

```
        e.printStackTrace(); // diagnóstico
        System.err.println("Datos inválidos: " + e.getMessage());
    }
}
```

Excepciones: Evento anómalo que ocurre durante la ejecución de un programa (runtime). Interrumpe el flujo normal de instrucciones.

Validaciones → para **errores esperables** y de **entrada de usuario** (faltan campos, formato de email, rango inválido). Se manejan **sin** romper el flujo normal.

Excepciones → para **situaciones excepcionales/imprevistas** donde **no se puede continuar** con la operación (invariantes rotas, recursos externos fallan, estado inconsistente, contratos incumplidos).

No usar excepciones como flujo normal de control. No conviene abusar de múltiples **catch** genéricos vuelve el código más difícil de leer y mantener

¿El usuario puede cometer este error normalmente? → **Validación**, sin excepción.

¿Seguir pondría el sistema en estado inválido? → **Excepción** (fail-fast).

¿Es un fallo externo (IO/DB/red)? → Probablemente **excepción**, manejar arriba (retry, fallback o informar).

¿Vas a usar el "error" como rama normal de lógica? → **No uses excepción** (usa if/Result).

¿Necesitas reportar **varios** errores de una pasada? → **Validaciones** (acumula mensajes)

Ejemplo: dividir entre 0, acceder fuera de un array, `ArithmeticException`, formato inválido.

// si se divide entre 0 en consola da `arithmeticexception`

//evitamos el error con lo siguiente

```
    int resultado = 0;
```

// resultado fuera como variable global

```
    try {
```

```
        resultado = num1 / num2;
```

```
    } catch (ArithmeticException e) {
```

//`arithmeticException` el error y `e` puede ser cualquier letra que funciona como una variable

```
        System.out.println("no es posible dividir entre 0");
```

```
        System.out.println(e.getMessage()); // / by 0
```

//se pueden poner mas `catch`

```
    } catch (ArrayIndexOutOfBoundsException e) {
```

```
        System.out.println("no puede almacenar datos en el arreglo");
```

// con este `catch` **captura todas la excepciones**

```
    } catch (Exception e) {
```

```
        System.out.println(e.getMessage());
```

```
    }
```

```
    System.out.println("el resultado es:" + resultado);
```

Java

```
in.close();
```

Checked IOException (comprobadas) → El compilador obliga a manejarlas con `try-catch` o declararlas con `throws`.

error: unreported exception IOException; must be caught or declared to be thrown.

try-catch-finally → bloque para manejar excepciones, atrapa la excepción y se define cómo reaccionar.

finally: bloque opcional que siempre se ejecuta (para liberar recursos).

Excepciones personalizadas: clase que extiende de `Exception`, con mensaje.

Ejemplo: `EmailFormatException`

En su propia clase `EmailFormatException` New > Class > `EmailFormatException` extends `Exception`):

```
public class EmailFormatException extends Exception {  
    public EmailFormatException(String message) {  
        super(message);  
    } // constructor  
} // class
```

En la clase donde esta la info (ej. persona)

// export desde aqui

// **método setter** `setEmail(String email)`, para asignar el valor del atributo `email`.

// `throws EmailFormatException` → método que **avisa** que puede lanzar una excepción de tipo `EmailFormatException` si algo sale mal. Obliga a hacer uso de `setEmail(...)` manejar esa excepción con `try-catch`.

```
public void setEmail(String email) throws EmailFormatException {
```

//Si el `email` es `null`, no intenta validarlo.

```
    if (email == null) {
```

//Asigna un **valor por defecto** (`sincorreo@dominio.com`).

```
        this.email = "sincorreo@dominio.com";
```

//`return`; hace que el método termine ahí mismo.

```
        return;
```

```
    }
```

// validación de correo, si en main no cumple sale el else en consola

```
    Pattern regex = Pattern.compile("[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
```

//`Pattern` → define un **patrón** (regex) representa estructura básica de un correo

```
    Matcher m = regex.matcher(email);
```

//`Matcher` → herramienta que compara el `email` recibido contra ese patrón

```
    if (m.matches()) {
```

```
        this.email = email;
```

```
    } else {
```

```
        this.email = "sincorreo@dominio.com";
```

Java

```
// desde aqui clickeando emailformatexception importo y hago el throw
    throw new EmailFormatException("El correo [" + email + "] no tiene un formato
válido");
    } // if
} // setemail
```

En main

```
public static void main(String[] args) {
    Persona p = new Persona();
    try {
        p.setEmail("correo_mal_escrito.com");
    } catch (EmailFormatException e) {
        System.out.println("Error capturado: " + e.getMessage());
    }
}
```

throw → lanza una excepción en tiempo de ejecución.

Se usa dentro del método para lanzar una excepción concreta.

Solo lanza **una instancia**.

```
throw new EmailFormatException("mensaje");
```

throws → declara que el método puede lanzar una excepción, es como un aviso.

Se usa en la firma del método para indicar que puede lanzar una excepción.

```
public void setEmail(String email) throws EmailFormatException
```

Unchecked (no comprobadas / Runtime) → Heredan de RuntimeException. El compilador **no obliga** a manejarlas, pero pueden ocurrir en ejecución.

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

h

Herencia:

Extender una clase existente para crear una subclase más especializada.

La nueva clase puede incluir métodos y **atributos** propios para completar su función.

Una subclase puede redefinir **métodos** heredados si no son final.

No se heredan los miembros privados.

No se pueden heredar **constructores**.

Utilizando la palabra reservada **super** podemos **acceder a atributos, métodos y constructores** de la clase padre desde la clase hija.

```
[modificador] class NomClase extends NomSuperClase {
```



Java

super.atributo: accede a un atributo de la clase padre.

super.metodo(): accede a un método de la clase padre.

super(): llamamos al constructor vacío del padre.

super(objeto): llamamos al constructor de copia de la clase padre.

super(param1, param2...): llamamos al constructor de parámetros del padre, con los parámetros que necesita.

Implementación

Se realiza utilizando la palabra reservada **extends**, seguida del **nombre de la clase padre**.

Clase Padre (Base/Superclase): De la que se deriva otra clase. Es la clase a partir de la cual se van a crear especializaciones.

Clase Hija (Derivada/Subclase): Se deriva de otra clase. Es la especialización.

Herencia única: Clase derivada se crea a partir de una sola clase base.

Herencia jerárquica: Clase heredada por muchas subclases.

Herencia multinivel: Clase derivada se crea a partir de única clase base, y esta clase derivada se convierte en la clase base para la nueva clase. (Padre, hija, neta)

Herencia múltiple: Clase derivada se crea a partir de más de una clase base. Java no admite el tipo de Herencia Múltiple con clases, es decir, no se puede extender o heredar de dos clases a la vez, pero se puede lograr haciendo uso de Interfaces.

Sobrecarga Overloading: Mismo nombre, distinta firma → distinta forma de usarse (distintos parámetros).

Overloading → compilación.

Consiste en tener métodos con el mismo nombre, pero con diferente lista de parámetros (cantidad, tipo u orden). **Firma del método** (nombre + parámetros)

El **tipo de retorno** puede variar

Pasa **en la misma clase** (aunque también aplica en herencia).

Se decide **en tiempo de compilación** (Java sabe cuál usar según los parámetros).

En main

```
Persona p1 = new Persona();
Persona p2 = new Persona("Ana");
Persona p3 = new Persona("Luis", 30);

p1.saludar();           // Hola!
p2.saludar("¿Cómo estás?"); // Hola! ¿Cómo estás?
```


Java

Sobreescritura Overriding: redefinir método heredado con la misma firma → redefinido en una clase hija (herencia).

Overriding → ejecución.

Pasa en herencia (clase padre → clase hija).

Un método de la clase hija (subclase) **tiene la misma firma** (nombre, parámetros) que el método de la clase padre (superclase), pero le da una **nueva implementación**.

Se usa la anotación `@Override` para marcarlo.

Polimorfismo: al usar un objeto de tipo padre que apunta a una instancia de la clase hija, se ejecuta el método sobrescrito de la hija.

Se decide **en tiempo de ejecución** (qué método corre depende del objeto real).

Super - Si se sobreescribe un método pero igual se quiere usar la versión del padre, se llamar con `super.metodo()`

Diferencias

Características	Sobrecarga Overloading	Sobreescritura Overriding
¿Dónde ocurre?	Misma clase o herencia	Herencia entre clase padre e hija
Firma (nombre+parámetros)	Misma nombre, distinta firma (parámetros es decir cambiar número, tipo u orden)	Misma firma (parámetros idénticos)
	Se elige en compilación	Se elige en ejecución
Herencia	No requiere herencia	Requiere herencia
Retorno	Puede ser distinto	Debe ser igual (o covariante)
Uso común	Flexibilizar creación de objetos, utilidades	Polimorfismo: redefinir comportamiento
Anotación	-	@Override

i

Identificador: nombres (únicos) que el programador da a los componentes que se están manejando en un programa, es decir, el nombre de las clases, métodos y variables.

Reglas:

- Nombres significativos.
- Se recomienda empezar por una letra, aunque después se pueden emplear números.
- No se permite el uso de caracteres especiales como: espacio en blanco, +, %, *, etc.
- Comienzan con mayúsculas y cuando son nombres compuestos cada palabra comienza con mayúsculas. Ej. `public class RecetaBizcocho`.
- Java distingue entre mayúsculas y minúsculas. Ej. `bizcocho` es distinto que `Bizcocho`.

Java

- Los nombres de constantes se escriben todo con mayúsculas y si el nombre es compuesto se usa el carácter de subrayado para separar las palabras. Ej. TIEMPO_HORNEADO, pero para el nombre de la clase, métodos y atributos, no se usa el carácter de subrayado.
- No pueden coincidir con palabras reservadas/clave

Iterator: Interface. Permite recorrer una colección

Valida si existen elementos

Recorre uno a uno los elementos

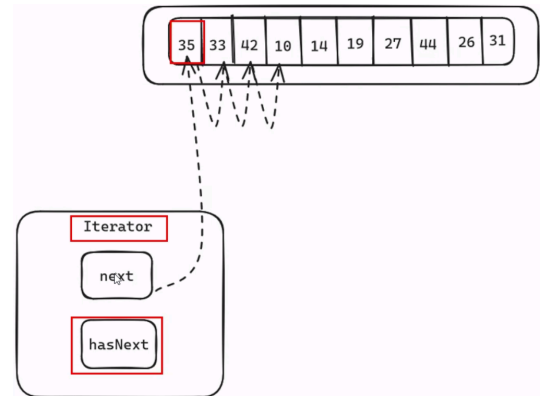
No aplica para mapas

hasNext() → devuelve true si todavía hay elementos que recorrer en la colección. Si ya no hay un valor mas dará false.

next() → devuelve el siguiente elemento de la colección y avanza el cursor.

```
Iterator<String> it = lista.iterator();
while(it.hasNext()){
    System.out.println(it.next());
}
```

hasNext() evita que el bucle intente leer cuando ya no hay elementos.



j

Java: Lenguaje de programación orientado a objetos (POO). Los datos son representados por diferentes objetos que proporcionan una biblioteca estándar rica, testeada, fiable y reutilizable.

1. Código Java (.java)
2. Compilador Java JAVAC
3. Bytecode (.class)
4. Máquina virtual java JVM

Package **ejemplos;** → Paquete

//A continuación la clase → Comentario

class **Ejemplo1**{ → Clase

```
public static void main(String[]
args) {
    System.out.println("Hola Mundo");
    System.out.println ("Esto es un
ejemplo");
}
```

→ Método
main

→ Bloque de
sentencias

Java

5. Lenguaje máquina

Incluye características que protegen a los programadores contra errores no intencionados que pueden provocar problemas de integridad.

Se trata de un lenguaje **fuertemente tipado** y capaz de hacer validaciones en tiempo de compilación y ejecución.

Proporciona una gestión de memoria automática. Cuando se crea un objeto, se asigna automáticamente espacio de memoria para ese objeto.

El recolector de basura Java realiza un seguimiento de cuáles son los objetos que la aplicación ya no necesita y recupera la memoria que ellos ocupan, evitando el problema de **fugas de memoria**.

Entorno multihilo, cada hilo (flujo de control del programa) representa un proceso individual dentro del programa, teniendo la capacidad de realizar múltiples tareas simultáneas.

Puede ejecutarse igualmente en cualquier otra plataforma o dispositivo. "write once, run anywhere".

Java SE: Standard Edition, Standalone, core, desktop, command line. JVM, JRE, JDK, SDK.

Java EE: Enterprise Edition, Service Side processing.

API application programming interface.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/module-summary.html>

Conceptos

	Qué es	Descripción	Qué hace
JVM	Java Virtual Machine	Máquina que ejecuta bytecode	Compila un archivo .java, lo transforma en .class (bytecode)
JRE	Java Runtime Environment	Entorno para ejecutar Java JVM + Librerías estándar de Java (API)	Lo que necesita un usuario final para ejecutar programas Java, pero no permite desarrollarlos.
JDK	Java Development Kit	Kit para desarrollar en Java JRE + herramientas para programar (compilador javac, javadoc, jar, jbd..)	Lo que necesita un desarrollador para escribir, compilar y ejecutar aplicaciones Java.
SDK	Software Development Kit	Término genérico que refiere a conjunto de herramientas para desarrollar software, en cualquier tecnología.	Ej. JDK es un SDK específico para Java.
LTS	Long-Term Support	Versión con soporte extendido	Actualizaciones de seguridad y correcciones de bugs por varios años (normalmente 8+ años).

I

Java

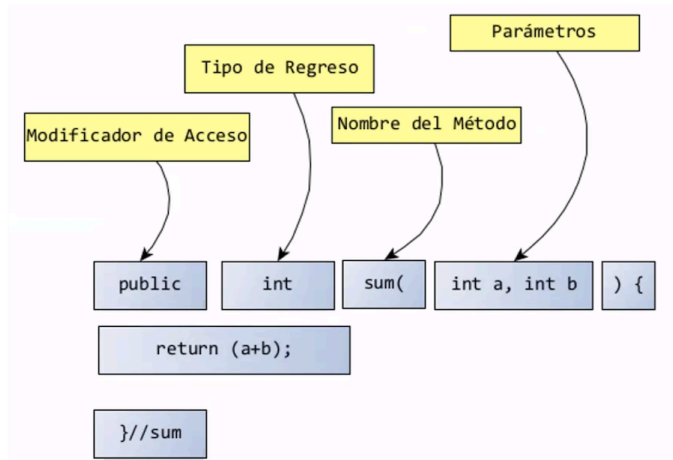
m

Método: Comportamiento de una clase. Grupo de declaraciones o sentencias que realizan una tarea específica. Se puede llamar a un método tantas veces como lo necesitemos.

Cabecera de dicho método + **cuerpo** (grupo de sentencias) definido entre llaves "{}".

Cabecera

- Modificadores (opcional).
- Tipo de retorno (void en caso de no retornar nada).
- Nombre: dar un nombre al método (se recomienda sea un verbo camelCase) para poder ser invocado o usado cuando lo necesitemos
- Lista de parámetros: Pasar información específica cuando sea necesario para la ejecución del método. Se separa por comas y se indica, por cada parámetro, el tipo y el nombre.



Método main: Método de entrada, inicia la ejecución de cualquier programa en Java. Siempre debe incluir modificadores: `public` (el método será visible para todas las clases en todas partes) y `static` (método estático, se puede llamar o usar desde cualquier clase sin necesidad de crear una instancia de la clase en la que se ha creado dicho método).

No puede retornar un valor como resultado, debe indicar el valor `void` como retorno.

Su parámetro de entrada siempre será un array de tipo `string` que debe aparecer obligatoriamente como argumento de este método.

```
public static void main(String args[])
```

```
public static void main(String args[]) {}  
public static void main(String[] args){}  
public static void main(String... args){}
```

Modificadores: pueden ser adicional, precedidos de uno o varios.

- **De acceso:** Especifica las restricciones de acceso o el alcance de una clase, método o variable.
 - **public:** Cuando se declara un método como público, se está permitiendo que desde cualquier otra clase pueda referenciarse directamente.
 - **private:** Si se declara un método como privado, no podrá ser invocado desde ninguna otra clase.
 - **protected:** Los métodos así declarados son accesibles directamente desde la propia clase, de las subclases de esta (herencia) y las clases que se encuentran dentro del mismo paquete.

Java

- **De no acceso:** No cambian el alcance de la clase, método o variable, sino que les proporciona funciones adicionales
 - **static:** Definir métodos que pertenecen a la clase y no a la instancia de la clase. No es necesario crear una instancia u objeto de la clase para poder acceder o utilizar ese método, simplemente se pueden invocar por su nombre siempre que se use en la misma clase donde se defina o, si se usa desde otra clase, anteponiendo el nombre de la clase donde se ha creado: nombreClase.nombreMetodo.
 - **final:** No puede ser sobrescrito por las subclases de la clase a la que pertenece. Solo los métodos static pueden ser final.
 - **abstract:** No permite instanciar la clase (crear un objeto) solo heredar. No tiene una implementación, sino que la implementación vendrá dada por las subclases de la clase a la que pertenece. Es un método incompleto, no tiene cuerpo o implementación, por eso no tiene llaves "{}" y termina con punto y coma: `public abstract void saludar();`

Retorno: uso de **void** o **return**. Dentro de un método con retorno de datos, las líneas de código que existan después del return no se ejecutarán

o

Objeto: Instancia de una clase (materialización)

Object es la raíz de la jerarquía de clases de Java

Metodos heredados de Object:

Método equals: comparar dos objetos de una clase.

Comparar referencias; las direcciones de memoria donde se ubican dos objetos, se puede usar directamente el operador `==`. si no se sobrescribes, compara referencias igual que en el caso de comparar variables, este operador compara el valor.

Comparar contenido o atributos: (uno, varios o todos sus atributos)

Sobrescribir para definir cómo debe actuar a la hora de comparar los dos objetos.

Método toString(): Convierte objeto a texto.

Siempre lleva @Override para indicar sobrescribir el toString de Object.

Antes de invocarlo, hay que sobrescribirlo para especificar qué componentes (atributos o métodos) de la clase se muestran y en qué formato.

En java

```
public class Persona {  
    String nombre;  
    int edad;  
  
    public Persona(String nombre, int edad){  
        this.nombre = nombre;  
    }  
}
```

Java

```
        this.edad = edad;
    }

    @Override
    public String toString(){
        return "Nombre: " + nombre + ", Edad: " + edad;
    }
// [ ] en son puramente decorativos para delimitar los atributos. No afectan la lógica, solo son texto fijo que se concatena.
}
```

En main

```
Persona p1 = new Persona("Ana", 21);
System.out.println(p1);
```

La mayoría de las clases de la API de Java, tienen sobrescrito este método para mostrar la información de acuerdo con el objeto que se maneja.

Todos los objetos en Java heredan de la clase base `Object`, y esa clase tiene un método llamado `toString()`.

Los métodos `println()` o `print()` pasan un objeto, este llama automáticamente al método `toString()` de ese objeto.

Operador: Permiten realizar operaciones entre los tipos de datos básicos.

- **De asignación:** Asignar un valor a una variable, usa el signo igual (=).
- **Aritmético:** Realizar operaciones matemáticas. El operador suma (+) puede usarse también para cadenas de texto: `String saludo = a + b`, donde `a="Buenos "` y `b="días."`, ahora `saludo` es igual a `"Buenos días."`
- **Aritméticos combinados:** Combinar un operador aritmético con el operador de asignación (`+=`).
- **Unario:** Actúan sobre un único operando y sirven para incrementar y decrementar en uno el valor. (`++`, `--`) `Variable=++contador`
- **Relacional:** Realiza comparaciones entre tipos primitivos con resultado `true` o `false`. (`==`, `<`).
- **Lógicos:** Compara variables o expresiones y el resultado será `true` o `false`. (`||`, `&&`).
- **Lógico ternario:** Utiliza la lógica de la condicional, asigna un valor a una variable dependiendo del resultado de una comparación. En función del resultado de la expresión lógica, devolverá un valor u otro. `(?) (condición)?valor1:valor2;`

Java

Outline: Esquema (Outline view) entornos de desarrollo integrados (IDEs) como Eclipse
Estructura general de una clase Java

1. Package (si existe).
2. Imports.
3. Declaración de la clase.
4. Atributos (variables de instancia).
5. Constructores.
6. Métodos (getters, setters, lógicos).
7. Método `main` (si existe, casi siempre hasta abajo).

p

Palabras clave/reservadas: definidas de manera exclusiva por el lenguaje de programación para un propósito específico.

Existen palabras reservadas para:

Datos primitivos

Palabra clave	Descripción	Rango
boolean	true/false.	
byte	número entero de 8 bits.	-128 a 127 entero pequeño
char	dato valor unico de de 16 bits.	Entre comillas simples 'a'
float	número real en coma flotante 32 bits.	$\pm 3.4e38$ (menor precisión que double) Termina en f (ej. 3.14f)
double	número real en coma flotante 64 bits.	$\pm 1.7e308$
short	número entero 16 bits.	-32,768 a 32,767 entero mediano
int	número entero 32 bits.	± 2 mil millones entero más común
long	número entero 64 bits.	Agrega L al final (ej. 123L)

Estructuras de control

	Palabra clave	Descripción
Condicionales	if	instrucciones de control condicionales simples (if) o múltiples (else if).
	else	instrucción de control condicional doble que indica qué hacer en caso de que no se cumpla la condición del (if) o el (else if).
	switch	instrucción de control condicional múltiple.
	case	indicar cada caso de una instrucción de control (switch).
	default	indica el caso por defecto de una instrucción de control (switch).

Java

Bucles	for	escribir bucles que se repiten un número determinado de veces.
	while	escribir bucles que se repiten mientras una condición sea cierta. La condición se valida al principio.
	do	escribir bucles (do while) que se repiten mientras una condición sea cierta. La condición se valida al final.
Salto/retorno	break	interrumpe la ejecución de un bucle o de una instrucción.
	continue	interrumpe la ejecución de un bucle, pero permite seguir realizando interacciones.
	return	interrumpe la ejecución del método y puede devolver un valor de retorno.
Excepciones	try	indica un bloque de código donde se quieren atrapar excepciones.
	catch	cláusula de un bloque (try) donde se especifica una excepción.
	finally	permite especificar un bloque de código que siempre se ejecutará, se produzca o no una excepción.
	throw	permite lanzar una excepción.
	throws	indica las excepciones que un método puede lanzar.

Modificadores

Palabra clave	Descripción
static	indica que un elemento es único en una clase y que se puede acceder al él sin tener que crear una instancia u objeto de la clase.
public	indica que un elemento es accesible desde cualquier clase.
private	indica que un elemento es accesible únicamente desde la clase donde se ha definido.
abstract	definir clases y métodos abstractos.
final	indica que una variable no se puede modificar, un método no se puede redefinir o una clase no se puede heredar.
protected	indica que un elemento es accesible desde la clase donde se ha definido, subclases de ella y otras clases del mismo paquete.
transient	especificar que un atributo no sea persistente.
volatile	indica que el valor de un atributo que está siendo utilizado por varios hilos esté sincronizado.
native	indica que un método está en un lenguaje de programación dependiente de la plataforma.
synchronized	indica que un método o bloque de código es atómico.

Estructura de datos

Palabra clave	Descripción
class	definir una clase.
import	importa una clase que se encuentra en otro paquete o en la API de Java .

Java

extends	permite indicar la clase padre de una clase.
interface	declarar una interfaz.
package	permite agrupar un conjunto de clases e interfaces.
implements	definir la/s interfaces de una clase.

Otros

Palabra clave	Descripción
super	permite invocar a un método o constructor de la superclase.
const	usaba para la definición de constantes, pero ya no se utiliza, ahora se usa (final).
this	referenciar al objeto e invocar al constructor.
new	crear un objeto nuevo de una clase.
assert	afirmar que una condición es cierta.
override	sobrescribir un método.
instanceof	permite saber si un objeto es una instancia de una clase concreta.
void	dato vacío (sin valor).
enum	definir tipos de datos enumerados.
goto	se usaba para moverse de un punto a otro del código, pero ya no se utiliza.
strictfp	indica que se tienen que utilizar cálculos en coma flotante estricto.

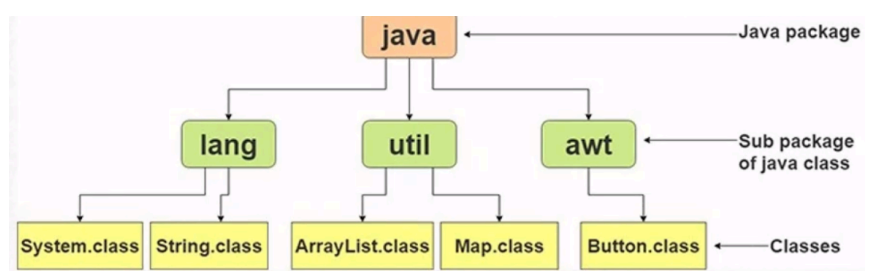
Paquete: forma de organizar las clases (indica la carpeta o directorio dónde está guardada la clase). Es la primera línea de código que debe estar en una clase Java.

Todas las clases que guardan cierta utilidad semejante se agrupan en un paquete. En una

clase solo se puede indicar un paquete (cuando no se le da un nombre específico, el compilador crea uno por defecto llamado default package).

patos.com -> .compatos

Es la primer línea de una clase



POO: Java es un lenguaje de **programación orientado a objetos**, intenta dar al código el mismo comportamiento y características que tienen los objetos del mundo real.

1. Herencia: las clases no están aisladas, sino que se relacionan entre sí, formando una estructura ramificada de clasificación en forma de un árbol jerárquico. Los objetos

Java

heredan las propiedades y el comportamiento de las clases a las que pertenecen. Permite extender una clase existente para crear una clase más especializada.

2. **Polimorfismo:** Permite enviar el mismo mensaje a objetos de diferentes clases, cada uno de ellos responde a ese mismo mensaje de modo distinto dependiendo de su diseño.
3. **Encapsulamiento:** Capaz de limitar o restringir el acceso a los atributos de una clase, o a los estados de un objeto, considerándolos "privados" y permitiendo tener un mayor control sobre ellos.
4. **Abstracción:** Proceso de interpretación y diseño. Implica enfocarse en las características importantes del objeto, ignorando todas las particularidades no esenciales. Permite identificar las características diferenciales que distinguen a un objeto de objetos de otro tipo.

Polimorfismo: concepto fundamental en la programación orientada a objetos, permite que objetos de diferentes clases sean tratados como objetos de una clase común.

Interface: define un contrato qué métodos deben implementar las clases. Obliga a que distintos objetos cumplan e implementen los mismos métodos mediante implement.

Métodos public y abstract por defecto.

Puede incluir default methods (con implementación) y static methods.

Ej.

Interfaz

```
public interface Calculable {  
    //metodos  
    double calcularArea();  
    double calcularPerimetro();  
    String getNombre();  
}
```

Java class

```
public class Rectangle implements Calculable {  
    private final String nombre;  
    private final double base, altura;  
  
    public Rectangle(String nombre, double base, double altura) {  
        this.nombre = nombre;  
        this.base = base;  
        this.altura = altura;  
    } //constructor
```

```
    //añadir getters, setters, toString según el caso para facilitar manejo de datos
```

Java

```
@Override public double calcularArea() { return base * altura; }
@Override public double calcularPerimetro() { return 2 * (base + altura); }
public String getNombre() { return nombre; }
} //metodos que se modifican en cada objeto mediante return (dando doble click sobre la palabra Rectangle)
```

Java Main (imprimir consola)

```
public static void imprimirCalculos(Calculable f) {
    System.out.println("Figura: " + f.getNombre());
    System.out.printf("Área: %.2f%n", f.calcularArea());
    System.out.printf("Perímetro: %.2f%n", f.calcularPerimetro());
}
```

Pasos:

1. Crear mas de una clase a aplicar la interfaz
2. Crear interfaz con los métodos
3. En cada clase (seguido de public class) agregar el implements + nombre de la interfaz
4. Añadir constructor, setter, getters, toString (no "setters" cuando tus campos son final)
5. En cada clase dar doble click al nombre de la clase para agregar métodos faltantes @override, y completar su respectiva información que se vera reflejada en main
6. En main declarar la interfaz en el método (Calculate t), hacer uso de polimorfismo

Precedencia de operadores

1. Paréntesis () [] .
2. Operadores unarios: ++, --, !, (cast) new
3. Multiplicación *, división /, módulo %
4. Suma + y resta -
5. Operadores relacionales: <, >, <=, >=
6. Igualdad: ==, !=
7. Operadores lógicos: && (AND), || (OR)
8. Operadores ternarios ?:
9. Asignación: =, +=, -=

s

Sentencias: unidad mínima de ejecución de un programa. Cada sentencia finaliza con punto y coma (;).

Condicionales:

IF

Ejecutarán las instrucciones en función de la condición definida.

Java

Condicional **if** (si...), y seguido indicamos la condición para que se ejecute la sentencia que va a continuación. Opcionalmente se puede añadir el bloque **else** (sentencias que sólo se realizan cuando no se cumple la condición if).

```
if (condición) {  
    // sentencias si la condición es verdadera  
}
```

Flujo

1. Evalúa la condición (expresión booleana → true o false).
2. Si es true, ejecuta el bloque correspondiente.
3. Si es false, pasa al siguiente **else if** o al **else**.
4. Después, continúa el programa normalmente.

- **if** (condición) → true → ejecuta bloque → fin.
- Si no → pasa a **else if** (condición2) → true → ejecuta → fin.
- Si ninguna cumple → ejecuta **else**.

Ejemplo

```
int nota = 85;  
  
if (nota >= 90) {  
    System.out.println("Excelente");  
} else if (nota >= 70) {  
    System.out.println("Aprobado");  
} else {  
    System.out.println("Reprobado");  
}
```

SWITCH..CASE

Instrucción de **múltiples vías** que permite ejecutar diferentes partes del código en función del valor de una expresión.

Versión clásica:

El valor de la **expresión** debe ser de tipo **char**, **byte**, **short**, **int** o **String** (desde Java 7). Cada **case** debe tener un valor único.

Después de cada caso se usa la palabra **break** para evitar que la ejecución “caiga” en el siguiente caso (*fall-through*).

Opcionalmente, se puede añadir el bloque **default** (solo se ejecuta si no se cumple ninguno de los casos anteriores).

```
switch (expresion) {  
    case valor1:
```

Java

```
        // Código a ejecutar
        break;
    case valor2:
        // Código a ejecutar
        break;
    default:
        // Código si no coincide ningún caso
    }
}
```

Versión flecha (Switch Expressions – Java 14+):

Puede **devolver un valor** directamente (se usa como expresión, no solo como bloque de código).

Usa la sintaxis `case valor ->` en vez de `case valor:`.

No necesita break: cada flecha es independiente.

No hay fall-through: nunca se pasa al siguiente caso por accidente.

Puede usarse para asignar el resultado a una variable o pasarlo directamente a un método como `System.out.println()`.

Requiere cubrir todos los casos posibles; si no, hay que poner `default`.

```
System.out.println(
    switch (numero) {
        case 1 -> "Lunes";
        case 2 -> "Martes";
        case 3 -> "Miércoles";
        case 4 -> "Jueves";
        case 5 -> "Viernes";
        case 6 -> "Sábado";
        case 7 -> "Domingo";
        default -> "Día inválido";
    }
);
```

Repetitivas o de bucle:

FOR

Se ejecutarán las sentencias repetidamente un número de veces, basándose en una condición. La condición se evaluará antes de cada iteración.

```
for (inicio ; condición ; inc/dec) {
    // sentencias
}
```

inicio: donde se inicializa una variable de control (ejemplo: `int i = 0`). Solo se ejecuta una vez, al inicio del ciclo.

condición: expresión booleana que se evalúa antes de cada iteración (`i < 10`).

Java

- Si es true, se ejecuta el bloque de sentencias.
- Si es false, el bucle termina.

inc/dec (incremento / decremento): Se ejecuta al final de cada iteración (i++, i--, i+=2, etc). Controla el avance de la variable del ciclo.

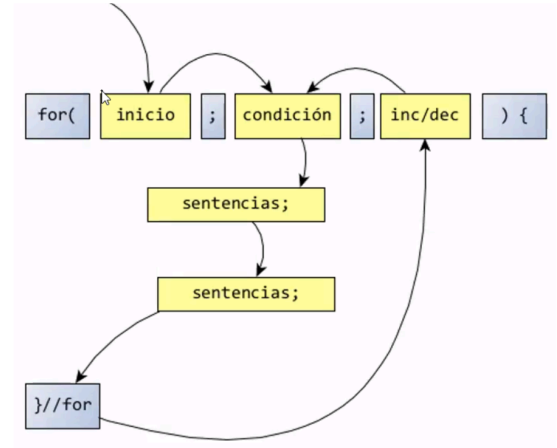
sentencias: bloque de código que se ejecuta mientras la condición sea verdadera.

Flujo

1. Se inicializa la variable (inicio).
2. Se evalúa la condición.
 - Si es verdadera → entra al bloque de sentencias.
 - Si es falsa → termina el bucle.
3. Ejecuta las sentencias.
4. Realiza el incremento o decremento (inc/dec).
5. Regresa a evaluar la condición otra vez.
6. Se repite hasta que la condición sea falsa.

Ejemplo

```
for (int i = 0; i < 5; i++) {  
    System.out.println("i = " + i);  
}
```



WHILE

Se ejecutarán y repetirán las sentencias mientras la condición sea verdadera.

Dentro del bucle, se han de actualizar los valores que se usan para la condición, si no se daría el caso de un bucle infinito. Si el resultado es false las instrucciones no se ejecutan y el programa seguirá ejecutándose por la siguiente instrucción a continuación del while.

```
while (condición) {  
    // sentencias  
}
```

Flujo

1. Evalúa la **condición** antes de entrar.
 2. Si la condición es true → ejecuta las sentencias.
 3. Vuelve a evaluar la condición.
 4. Si sigue siendo true, repite.
 5. Cuando es false, termina el bucle.
- si desde el inicio la condición es false, **el bloque nunca se ejecuta**

Ejemplo

```
int i = 0;  
while (i < 3) {
```

Java

```
        System.out.println("i = " + i);
        i++;
    }
```

DO WHILE

Igual que (WHILE), la diferencia es que primero ejecuta la condición y luego se evalúan las sentencias. **siempre ejecuta el bloque mínimo una vez**, incluso si la condición es falsa desde el principio.

```
    do {
        // sentencias
    } while (condición);
```

Flujo

1. Ejecuta el bloque de sentencias **al menos una vez**.
2. Evalúa la condición.
3. Si la condición es `true`, repite.
4. Si es `false`, termina.

Ejemplo

```
int j = 0;
do {
    System.out.println("j = " + j);
    j++;
} while (j < 3);
```

Anidadas: Incluyen instrucciones **if, for, while o do-while** unas dentro de otras, teniendo en cuenta que cada estructura que se anide debe estar cerrada dentro de la otra.

El bucle interno se repite completo por cada iteración del externo.

Si el externo corre n veces y el interno m veces $\rightarrow$ el total de iteraciones será $n \times m$.

Ejemplo

```
for (int i = 1; i <= 3; i++) {           // Bucle externo
    for (int j = 1; j <= 2; j++) {       // Bucle interno
        System.out.println("i=" + i + ", j=" + j);
    }
}
```

Java

Setters y getters: Atributos declarados como **private** necesitan métodos públicos conocidos como getters y setters; permiten manipular o modificar los valores de atributos o recuperar los valores de dichos atributos.

SET: Permite modificar los atributos privados.

GET: Permite a los objetos retomar los valores de sus atributos privados.

Atributo de tipo boolean, se usa la palabra "is" en lugar de "get".

```
public class Coche {  
  
    //declaración de atributos  
    private String color;  
    private double longitud;  
    private int plazas;  
  
    //declaración de método get que devuelve el color del coche, no lleva info en el  
    //paréntesis porque el return va a dar esa info  
    public String getColor() {  
        return color;  
    }  
  
    //declaración de método set que modifica el color del coche, lleva info en el paréntesis  
    //porque ocupa la información  
    public void setColor (String Rojo) {  
        this.color = rojo;  
    }  
}
```

System: clase de la biblioteca estándar de Java (`java.lang.System`) provee métodos y campos para interactuar con el sistema donde corre el programa.

System.in (input stream)→ Flujo de entrada estándar (teclado). Normalmente se usa con un `Scanner` para leer datos

```
Scanner sc = new Scanner(System.in);  
int numero = sc.nextInt();
```

System.out (output stream)→ Flujo de salida estándar (consola). Muestra datos en pantalla.

Java

t

Thread: hilo de ejecución concurrente dentro del misma aplicación. Permite que se ejecuten múltiples tareas de manera concurrente. Por lapsos de espera.

Creación

start(): inicia el hilo.

run(): contiene la lógica del hilo (no se invoca directo, sino con **start()**).

sleep(ms): pausa el hilo.

join(): hace que un hilo espere a otro.

isAlive(): verifica si sigue en ejecución.

Extender la clase Thread

```
class MiHilo extends Thread {  
    public void run() {  
        System.out.println("Ejecutando en un hilo");  
    }  
}
```

```
MiHilo t = new MiHilo();  
t.start(); // arranca el hilo
```

*En caso de herencia considerar constructor **super()**, ctrl espacio en run para generar **@override**.

Implementar la interfaz **Runnable** (más flexible, recomendado)

```
class MiRunnable implements Runnable {  
    public void run() {  
        System.out.println("Ejecutando con Runnable");  
    }  
}
```

```
Thread t = new Thread(new MiRunnable());  
t.start();
```

Java

Runnable: interfaz para definir tareas en hilos, funcional con un solo método
`void run();`

Permite heredar de otra clase (porque no extiende `Thread`).

Facilita el uso de *lambda expressions* ***función anónima** que se puede pasar como parámetro o asignar a una variable.

```
Thread t = new Thread() -> {  
    System.out.println("Hilo con lambda");  
};  
t.start();
```

p

print() → Imprime **sin** salto de línea.

println() → Imprime **con** salto de línea.

printf() → Imprime con formato (parecido a C).

Pruebas unitarias para asegurar calidad del código

Objetivo: verificar que métodos o clases funcionen como se espera. (Tener el resultado esperado definido para hacer múltiples pruebas).

Framework más usado: JUnit.

Click derecho en package explorer > New > Other > Java > JUnit > JUnitTestCase
*main Run As > JUnit (@test en la clase)

Estructura básica:

```
import static org.junit.Assert.*;  
import org.junit.Test;  
  
public class CalculadoraTest {  
  
    @Test  
    public void testSuma() {  
        Calculadora calc = new Calculadora();  
        assertEquals(5, calc.suma(2, 3));  
    }  
}
```

@Test: marca un método como prueba.

@BeforeEach: se ejecuta antes de cada test (inicialización).

@AfterEach: se ejecuta después de cada test.

@BeforeAll y @AfterAll: para inicialización/limpieza global.

Java

Asserts:

```
assertEquals(expected, actual)
assertTrue(condición)
assertFalse(condición)
assertNotNull(obj)
```

JUnit 5 (JUnit Jupiter)

`assertThrows`: probar que un método lanza una excepción esperada.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

@Test
void testDivisionPorCero() {
    Calculadora calc = new Calculadora();

    // Verifica que al dividir entre 0 se lance ArithmeticException
    assertThrows(ArithmeticException.class, () -> {
        calc.dividir(10, 0);
    });
}
```

`assertThrows(tipoExcepcion.class, () -> código)`: valida que se lance una excepción.

`@DisplayName("texto descriptivo")`: documenta la prueba con un nombre legible (en vez del nombre del método). Cuando se corre tests en consola o en un IDE.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

class CalculadoraTest {
    @Test
    @DisplayName("Suma de dos números positivos")
    void testSumaPositivos() {
        Calculadora calc = new Calculadora();
        assertEquals(5, calc.suma(2, 3));
    }
}
```

Java

```
@Test
@DisplayName("Lanza excepción al dividir entre cero")
void testDivisionPorCero() {
    Calculadora calc = new Calculadora();
    assertThrows(ArithmeticException.class, () -> calc.dividir(10, 0));
}
}
```

V

Variable: Objeto cuyo contenido puede variar durante el proceso de ejecución del algoritmo. Ej. datos para llevar a cabo una suma.

Declarar es identificar con un tipo y nombre a la variable. Es necesario declararlas antes de usarlas.

El tipo indica qué tipo de valores puede guardar.

El nombre permite acceder a su contenido y es necesario escribirlo en minúsculas.

<tipo dato> <nombre variable>;

Ej. Edad es un dato que puede variar durante el programa:

int edad;

Inicialización o asignación: se le da un valor por primera vez, antes de ser utilizada. A nivel de sintaxis se realiza usando el operador de asignación "=". Si no se inicia previamente, Java lo hace automáticamente.

Ej. valores numéricos Java les da un valor inicial de cero.

int edad = 0;

Asignar: previamente inicializada se le asigna un nuevo valor durante la ejecución. Dependiendo del tipo de variables se asignan de una manera u otra.

Tipo variable	Descripción	Ejemplo
Numéricos	se asignan directamente a la variable.	int edad = 20;
Caracteres	Usar la comilla simple. ' '	char genero = 'M';
Cadenas	Usar comillas dobles. " "	String nombre = "Rodrigo";
Booleanos	Usar verdadero o falso (true o false).	boolean casado = true;

Ámbito: Define su alcance de uso, es decir en qué secciones de código está disponible. Fuera de este ámbito, una variable no existe.

Local	sólo están visibles dentro del bloque de código donde se han declarado (dentro de un método, un bucle, un condicional...), cuando éste finaliza, deja de existir.
--------------	---

Java

Instancia o global	pertenecen a cada instancia concreta de la clase donde han sido declaradas y sólo pueden ser accesibles desde la propia instancia a la que pertenecen.
Clase o estática	pertenece a la clase, no a la instancia, y se puede acceder a ella desde cualquier parte de la clase donde han sido declaradas.